# Validate Subsequence

AlgoExpert · Easy · Arrays

# The problem

- Is `sequence` obtainable from `array` by deleting elements without reordering?

# Intuition

- A subsequence keeps order, so we just try to match the sequence's elements one by one while scanning the array.

# Brute force vs optimal

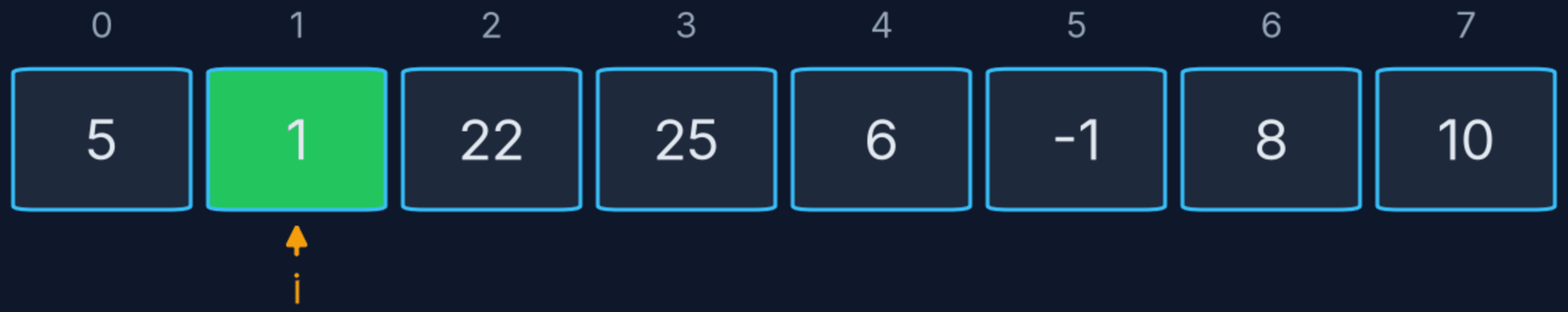
- Brute force: try every way to pick positions → exponential
- Optimal: one greedy pass, matching in order
- Greedy is safe: matching early never hurts

# Approach

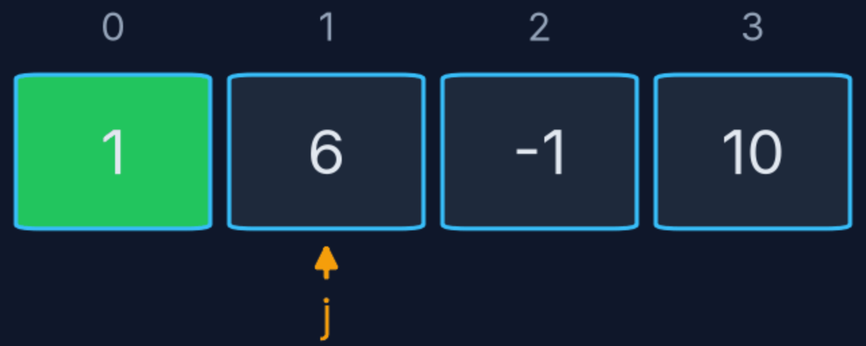
- 1. Keep an index into the sequence
- 2. Scan the array
- 3. Advance the index on every match
- 4. Valid if the index reaches the end

# Worked example

array



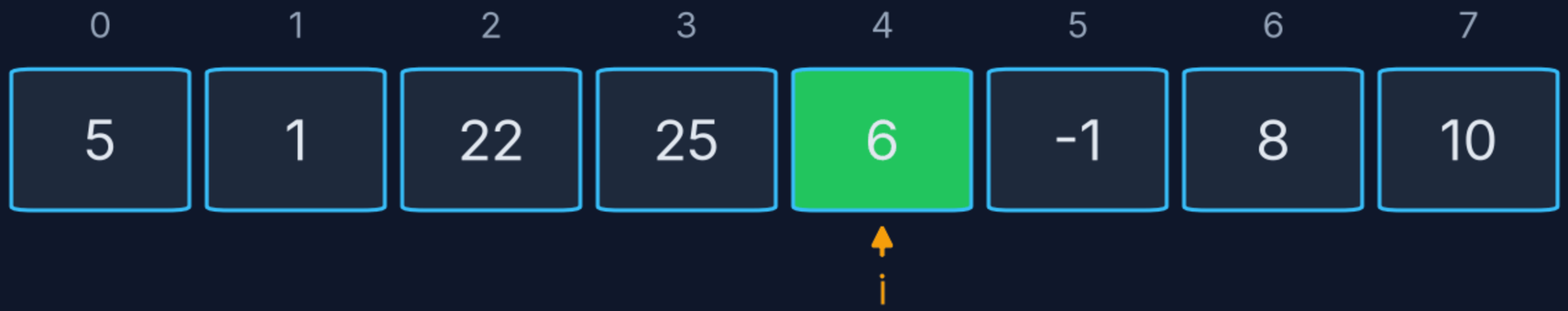

sequence



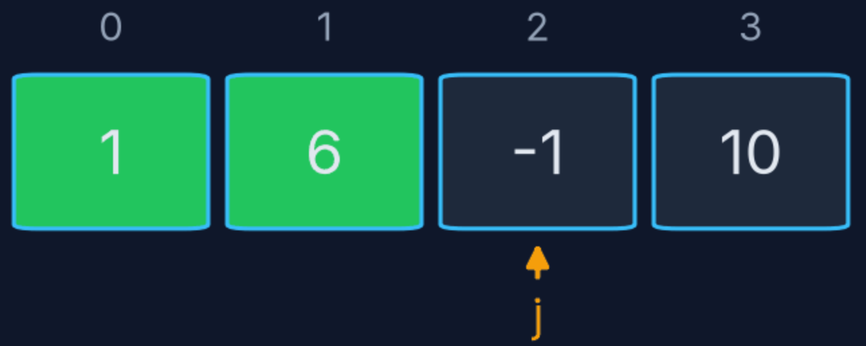

1 matches → advance j to 1

# Worked example

array



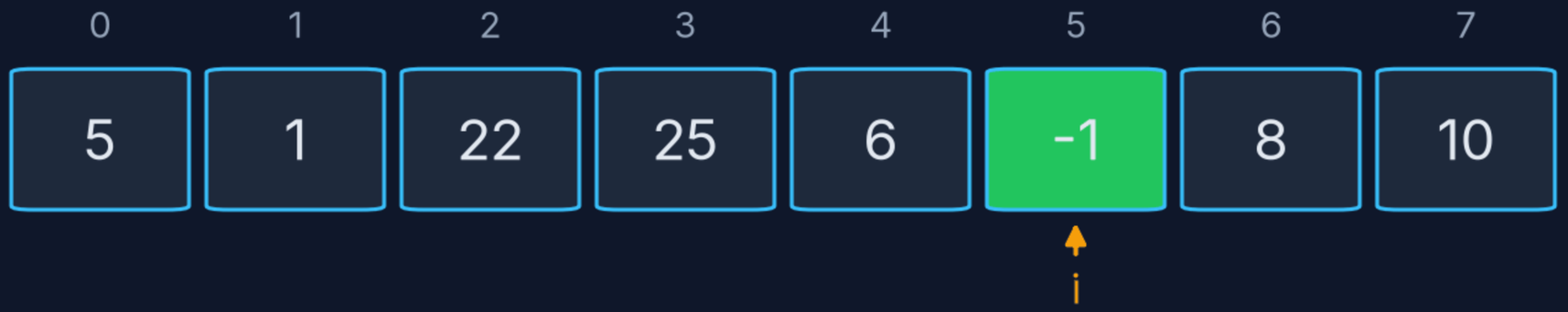

sequence



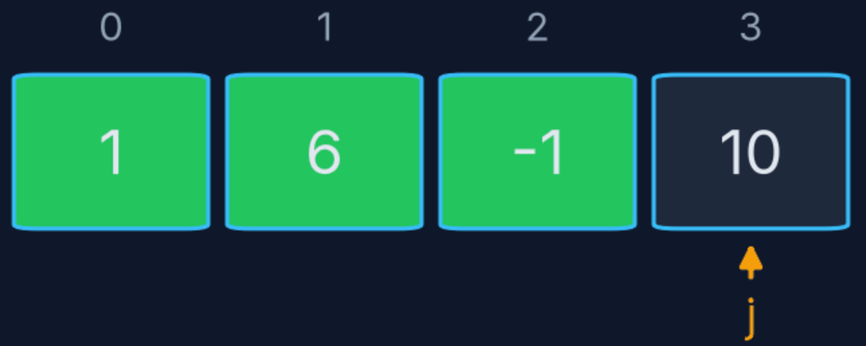

6 matches → advance j to 2

# Worked example

array



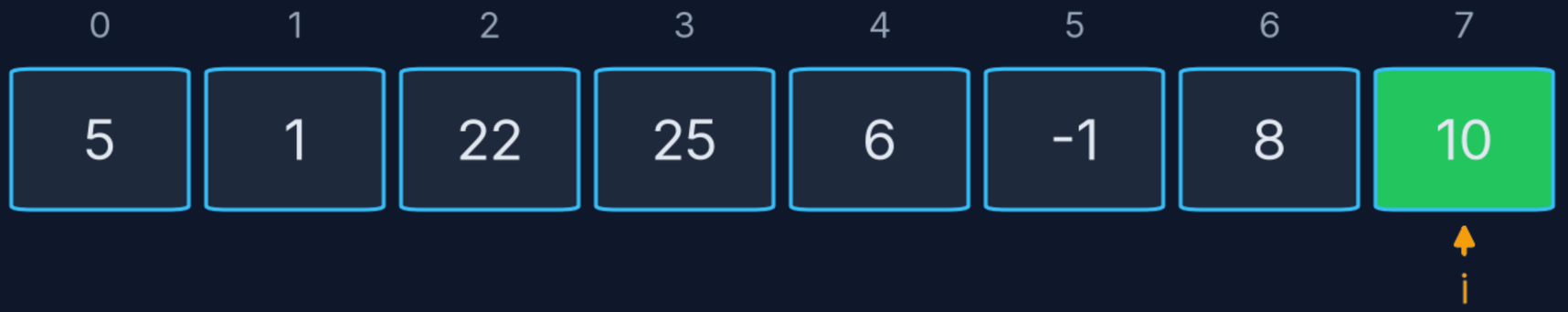

sequence



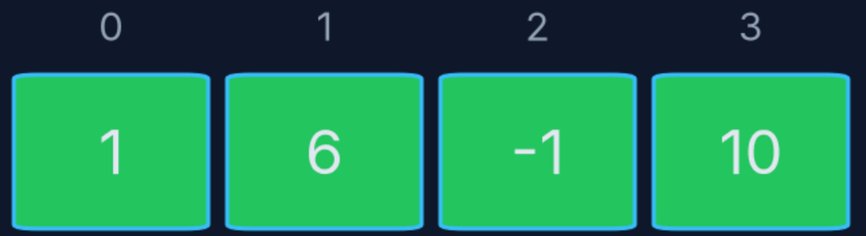

-1 matches → advance j to 3

# Worked example

array



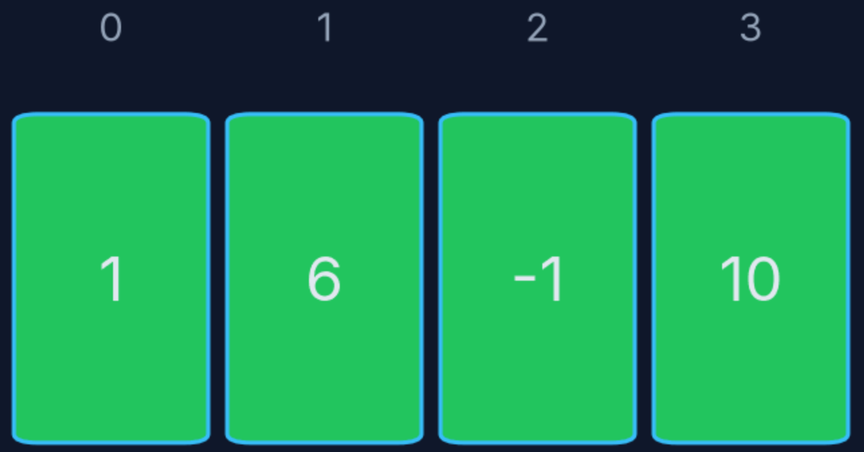

sequence




10 matches → advance j to 4

# Worked example

sequence




j = 4 = len(sequence) → valid

# Java solution

```java
public class ValidateSubsequence {
    public static boolean isValidSubsequence(List<Integer> array, List<Integer> sequence) {
        int j = 0;
        for (int i = 0; i < array.size() && j < sequence.size(); i++)
            if (array.get(i).equals(sequence.get(j))) j++;
        return j == sequence.size();
    }
}
```

# Complexity

- Time: O(n)
- Space: O(1)

# Edge cases & pitfalls

- Sequence longer than array → false
- Repeated values: each array element used once
- Single-element sequence

# Interview tips & follow-ups

- Clarify: subsequence ≠ subarray (no contiguity)
- Follow-up: check many sequences against one array

# Recap

- A subsequence keeps order, so we just try to match the sequence's elements one by one while scanning the array.
- Complexity: O(n)